LAPORAN PRAKTIKUM ALGORITMA DAN PEMROGRAMAN 2

MODUL 10

“PENCARIAN NILAI MAX MIN”

DISUSUN OLEH:

DYAH IMANSARI

109082500130

S1IF-13-02

DOSEN:

Wahyu Andi Saputra, S.Pd., M.Eng

PROGRAM STUDI S1 INFORMATIKA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2025/2026

DASAR TEORI

1. Konsep Dasar Pencarian Nilai Ekstrim

Pencarian merupakan proses fundamental dalam algoritma untuk menemukan elemen tertentu dalam sekumpulan data. Salah satu variasi pencarian yang penting adalah pencarian **nilai ekstrim**, yaitu mencari nilai terbesar (**maksimum**) atau nilai terkecil (**minimum**) dari sebuah koleksi data (seperti array).

Ide utama dari algoritma ini adalah memproses data secara sekuensial (berurutan). Algoritma akan menyimpan sebuah variabel sementara (biasanya disebut "nilai ekstrim sementara") yang menampung nilai terbaik yang ditemukan sejauh ini untuk dibandingkan dengan elemen-elemen berikutnya.

2. Algoritma Umum Pencarian Ekstrim

Secara umum, langkah-langkah untuk mencari nilai ekstrim adalah sebagai berikut:

1. **Inisialisasi:** Menjadikan data pertama pada kumpulan data sebagai nilai ekstrim sementara.
2. **Iterasi & Validasi:** Melakukan pengecekan dari data kedua hingga data terakhir.
3. **Update:** Jika ditemukan data yang lebih sesuai (lebih besar untuk maksimum, atau lebih kecil untuk minimum) dibandingkan nilai sementara, maka nilai sementara tersebut diperbarui dengan data yang baru dicek.
4. **Hasil:** Setelah semua data diperiksa, nilai sementara terakhir adalah nilai ekstrim yang valid.

3. Implementasi pada Bahasa Go

Dalam bahasa Go, pencarian ini biasanya dilakukan pada tipe data **Array** atau **Slice**. Perlu diperhatikan bahwa indeks array pada Go dimulai dari **0**.

a. Pencarian Berdasarkan Nilai

Pada metode ini, fungsi mengembalikan nilai (isi) dari elemen terkecil atau terbesar itu sendiri. Contoh logika untuk mencari nilai minimum:

Go

```
var min int = tabInt[0]
for j := 1; j < n; j++ {
   if min > tabInt[j] {
      min = tabInt[j]
```

```
    }
}
```

b. Pencarian Berdasarkan Indeks (Posisi)

Metode ini dianggap lebih efektif karena dengan mengetahui **posisi (indeks)** suatu elemen, kita tidak hanya mendapatkan nilainya, tetapi juga mempermudah akses ke data lain jika array tersebut berisi tipe data terstruktur (*struct*). Contoh logika pencarian indeks minimum:

```
Go
var idx int = 0
for j := 1; j < n; j++ {
    if tabInt[idx] > tabInt[j] {
        idx = j
    }
}
```

4. Pencarian pada Tipe Data Terstruktur (Struct)

Pencarian ekstrim juga sangat berguna pada data yang lebih kompleks, seperti array dari sebuah struct. Misalnya, dalam data mahasiswa, kita bisa mencari siapa yang memiliki IPK tertinggi.

Dengan menyimpan **indeks** dari mahasiswa yang memiliki IPK tertinggi, kita dapat mengakses seluruh atribut identitas mahasiswa tersebut (Nama, NIM, Kelas, dll.) melalui referensi indeks tersebut, contohnya: T[idx].nama atau T[idx].nim.

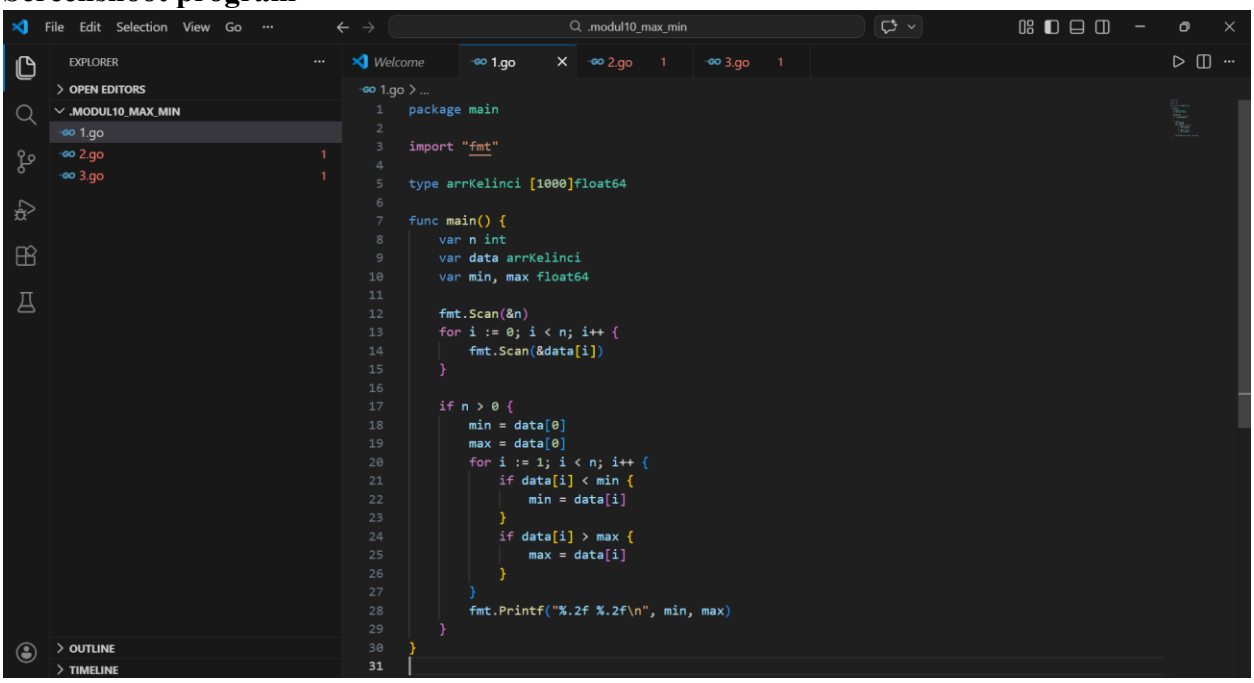
LATIHAN

1. **Latihan 1**

Source Code

```
package main

import "fmt"

type arrKelinci [1000]float64

func main() {
    var n int
    var data arrKelinci
    var min, max float64

    fmt.Scan(&n)
    for i := 0; i < n; i++ {
        fmt.Scan(&data[i])
    }

    if n > 0 {
        min = data[0]
        max = data[0]
        for i := 1; i < n; i++ {
            if data[i] < min {
                min = data[i]
            }
            if data[i] > max {
                max = data[i]
            }
        }
        fmt.Printf("%.2f %.2f\n", min, max)
    }
}
```

Screenshoot program



Deskripsi program

Program ini dibuat untuk mencari berat kelinci terkecil dan terbesar dengan cara menyimpan data masukan ke dalam array berkapasitas 1000, lalu membandingkan setiap isi data tersebut secara berurutan mulai dari data pertama yang dijadikan sebagai patokan nilai ekstrim sementara. Cara kerjanya adalah dengan melakukan pengecekan satu per satu di dalam perulangan, di mana variabel min akan diperbarui jika ada data yang lebih kecil dan variabel max akan diperbarui jika ada data yang lebih besar, hingga akhirnya program menampilkan kedua nilai tersebut sebagai hasil berat paling ringan dan paling berat.

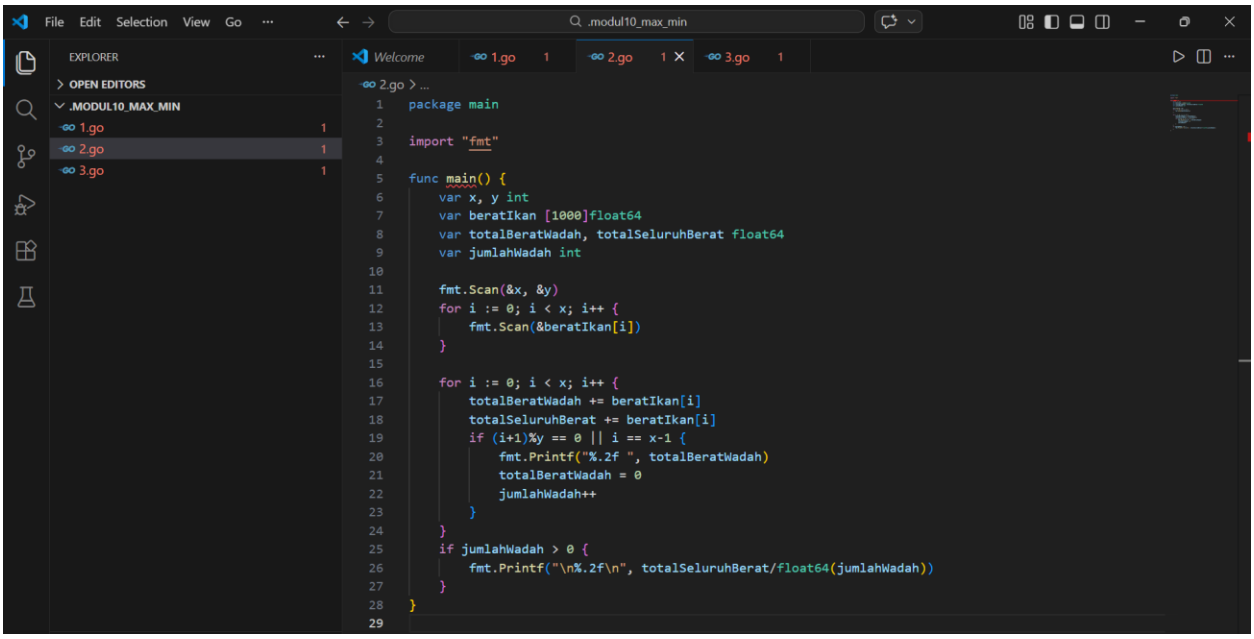
2. **Latihan 2**

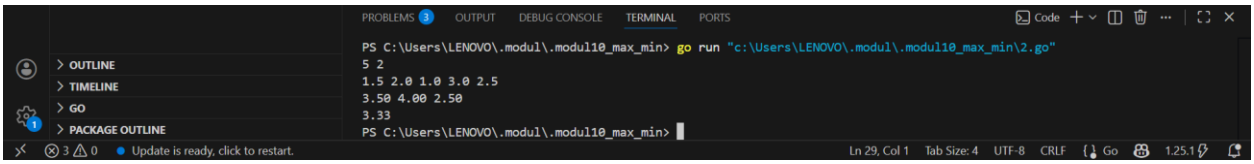
Source Code

```
package main

import "fmt"

func main() {
```

```
        var x, y int
        var beratIkan [1000]float64
        var totalBeratWadah, totalSeluruhBerat float64
        var jumlahWadah int

        fmt.Scan(&x, &y)
        for i := 0; i < x; i++ {
            fmt.Scan(&beratIkan[i])
        }

        for i := 0; i < x; i++ {
            totalBeratWadah += beratIkan[i]
            totalSeluruhBerat += beratIkan[i]
            if (i+1)%y == 0 || i == x-1 {
                fmt.Printf("%.2f ", totalBeratWadah)
                totalBeratWadah = 0
                jumlahWadah++
            }
        }
        if jumlahWadah > 0 {
            fmt.Printf("\n%.2f\n",
    totalSeluruhBerat/float64(jumlahWadah))
        }
    }
```

Screenshoot program





Deskripsi program

Program ini bertujuan menghitung total berat ikan yang dibagi ke dalam beberapa wadah berdasarkan kapasitas tertentu serta menentukan rata-rata berat per wadahnya. Cara kerjanya adalah dengan menerima jumlah total ikan (x) dan kapasitas tampung setiap wadah (y), lalu menjumlahkan berat ikan secara berurutan hingga sebuah wadah mencapai batas kapasitasnya sebelum beralih ke wadah berikutnya. Di akhir proses, program akan menampilkan deretan total berat untuk setiap wadah yang terisi dan satu nilai riil yang merepresentasikan rata-rata berat keseluruhan dibagi dengan jumlah wadah yang digunakan.

3. Latihan 3

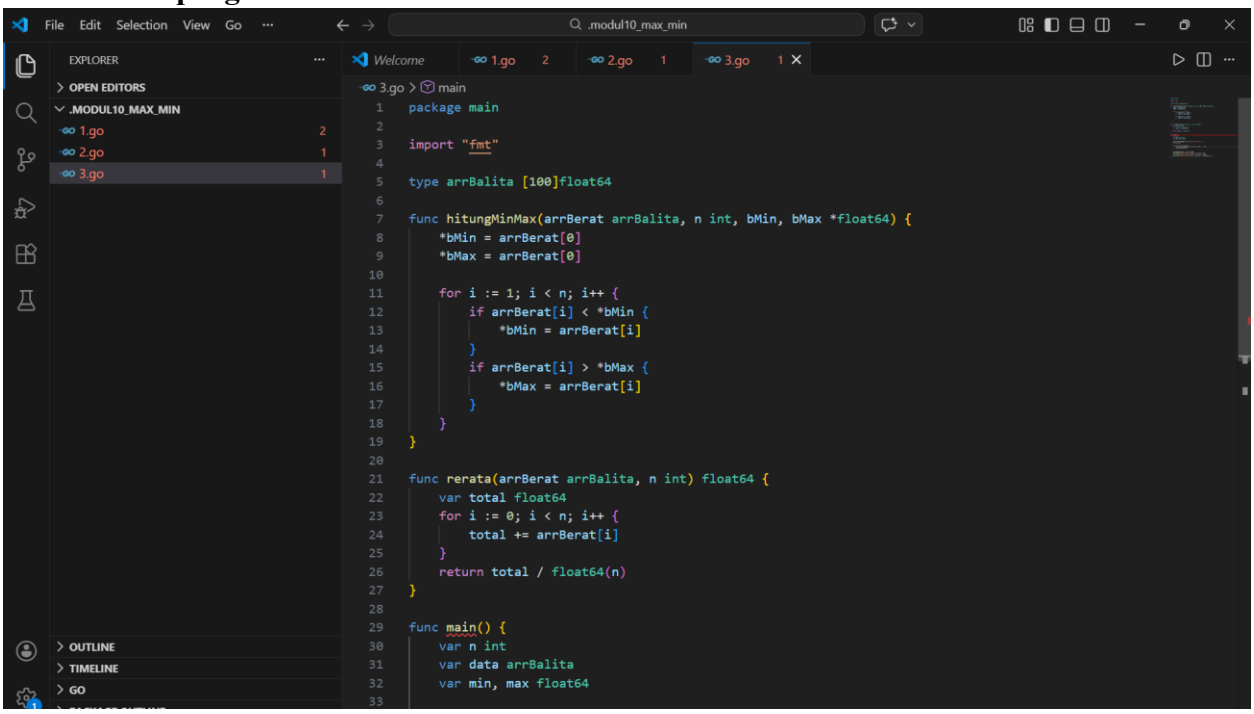
Source Code

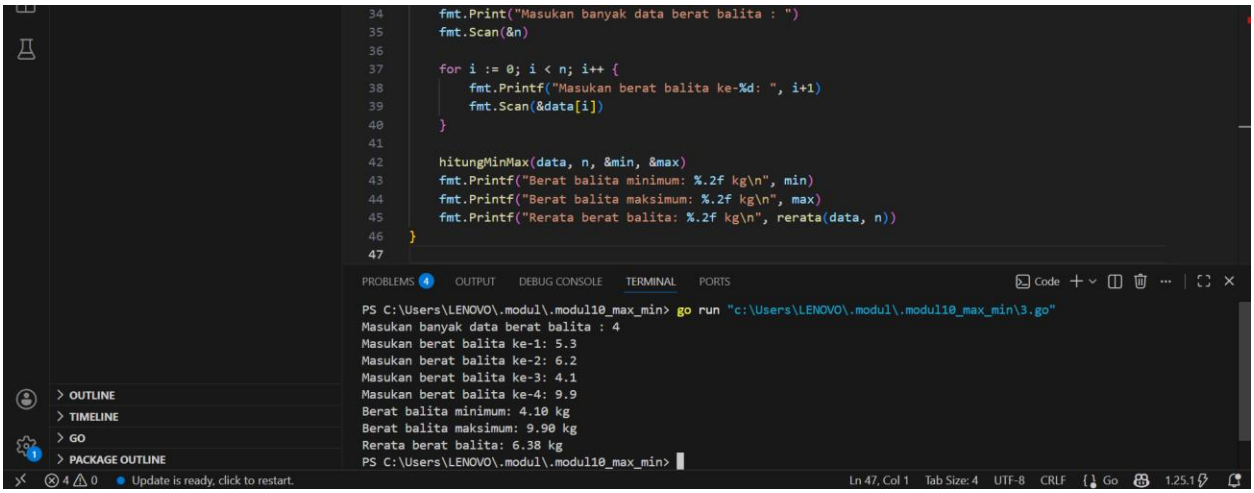
```
package main

import "fmt"

type arrBalita [100]float64
```

```
func hitungMinMax(arrBerat arrBalita, n int, bMin, bMax
*float64) {
    *bMin = arrBerat[0]
    *bMax = arrBerat[0]

    for i := 1; i < n; i++ {
        if arrBerat[i] < *bMin {
            *bMin = arrBerat[i]
        }
        if arrBerat[i] > *bMax {
            *bMax = arrBerat[i]
        }
    }
}

func rerata(arrBerat arrBalita, n int) float64 {
    var total float64
    for i := 0; i < n; i++ {
        total += arrBerat[i]
    }
    return total / float64(n)
}

func main() {
    var n int
    var data arrBalita
    var min, max float64

    fmt.Print("Masukan banyak data berat balita : ")
    fmt.Scan(&n)

    for i := 0; i < n; i++ {
        fmt.Printf("Masukan berat balita ke-%d: ", i+1)
        fmt.Scan(&data[i])
```

```
        }

        hitungMinMax(data, n, &min, &max)
        fmt.Printf("Berat balita minimum: %.2f kg\n", min)
        fmt.Printf("Berat balita maksimum: %.2f kg\n", max)
        fmt.Printf("Rerata berat balita: %.2f kg\n",
    rerata(data, n))
    }
```

Screenshoot program





Deskripsi program

Program ini mengimplementasikan pencarian nilai ekstrim dan perhitungan rata-rata pada data berat balita dengan menggunakan struktur subprogram berupa prosedur dan fungsi. Cara kerjanya adalah dengan menyimpan input berat badan ke dalam array berkapasitas 100, lalu memanggil prosedur hitungMinMax yang menggunakan *pointer* untuk memperbarui nilai minimum dan maksimum secara bersamaan, serta menggunakan fungsi rerata untuk mengalkulasi nilai rata-rata berat seluruh balita, sehingga hasil akhir ditampilkan dengan format dua angka di belakang desimal sesuai instruksi modul.

DAFTAR PUSTAKA

Tim Penyusun. (2026). *Modul Praktikum Algoritma dan Pemrograman 2*. Bandung: Fakultas Informatika, Telkom University.